

Task 1. For the fourth part of the project, we will explore the capabilities of Rascal as a language workbench and the power of the definition of languages using formal grammars. To do this, your task is to reuse the implementation of VeriLang from the previous project, but now we will target a different language (*i.e.*, not Java), Kotlin.

Starting from the definition of your project 3 (correcting the errors), run the project from the Kotlin runtime environment.

To do this, you should first extract the language (*i.e.*, the VeriLang AST) into a JSON file. Then you should create a Kotlin file (`VeriLangService.kt` to complete) that calls the JSON file to open it (an example of this file is provided with the project). Now, you should call the `VeriLangService` file from the Kotlin GUI provided with the project.

When running a verilang file, the interface should display:

- The working parser (should display ok for correct files, fail for incorrect files, displaying the Rascal error)
- A list with the modules that exists in the file
- other information that you may consider relevant

Take into account that the files given serve as an example, and you may need to adapt it to your own definition of the program.

Hand in a `.zip` file with the solution and document.